

PART 2 OF 3 • LEVEL UP

Advanced RAG

Query transforms, advanced indexing, hybrid search, re-ranking, Self-RAG, CRAG, GraphRAG, multimodal & production patterns



A self-study guide • Casual Hinglish style
Topics covered end-to-end with code, diagrams & intuition

1. Why Advanced RAG?

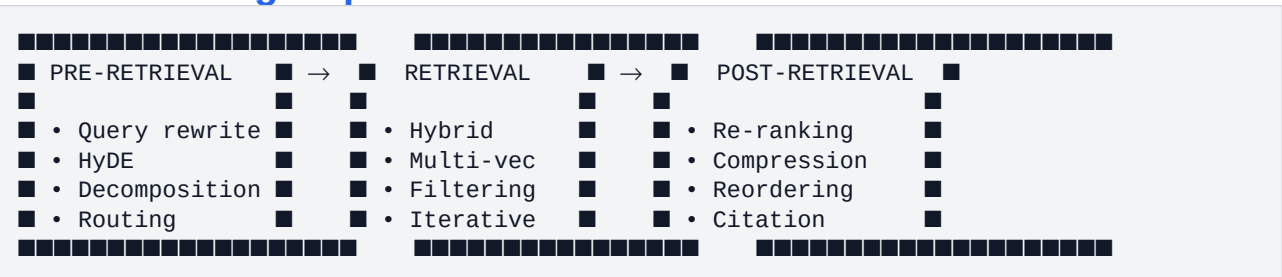
Base RAG (Part 1) ek toy example pe bahut accha kaam karta hai. Production me tum ye real problems face karte ho:

- **Ambiguous queries:** User chhota, lazy query likhta hai — 'cancel order' — kis order ka? Kab? Direct embedding match noisy hota hai.
- **Multi-hop reasoning:** 'CEO of company that acquired X in 2023?' — ek retrieval call kaafi nahi.
- **Diverse formats:** Tables, code, images mixed corpus me.
- **Outdated retrieval:** Top-k chunks semantically similar lagein but actually relevant nahi.
- **Redundancy:** Same fact 5 chunks me — context wasted.
- **Long-tail queries:** Embedding model out-of-domain hai → bad retrieval.
- **Verification:** Kya generated answer actually grounded hai?

■ KEY POINT

Advanced RAG = pre-retrieval, retrieval, aur post-retrieval har stage pe smart tricks. Goal: *maximize relevant info in LLM context, minimize noise.*

1.1 The 3-Stage Optimization Framework



Har stage me independent improvements possible hain. Yahi structure pe Part 2 chalega.

2. Advanced Chunking

2.1 Semantic Chunking

Idea: fixed-size se hatkar, **topic shifts** pe break karo. Adjacent sentences ke embeddings calculate karo; jaha cosine distance threshold se zyada ho, waha cut karo.

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

splitter = SemanticChunker(
    OpenAIEmbeddings(),
    breakpoint_threshold_type='percentile', # or 'standard_deviation'
    breakpoint_threshold_amount=95
)
chunks = splitter.create_documents([long_text])
```

- ■ Chunks naturally coherent topics ke around
- ■ Computationally expensive (embed every sentence)
- ■ Chunk sizes uneven — context budget management harder

2.2 Agentic Chunking

LLM ka use karke *logical units* identify karna. Model ko prompt karo — 'is text ko logical chunks me todo, har chunk ek self-contained idea ho'. Costly but high quality.

■ NOTE

Agentic chunking research-stage me hai. Production cost-prohibitive often — but expensive, infrequently-updated corpora (legal, medical) ke liye worth it.

2.3 Hierarchical Chunking

Multiple levels of granularity simultaneously store karo:

- Level 0: Full document (summary)
- Level 1: Section-level chunks
- Level 2: Paragraph-level chunks
- Level 3: Sentence-level chunks

Query time: chhote chunks pe search karo (precision), lekin parent context bhi pass karo (recall). Coverage Part 2.5 me — parent-child retrievers.

2.4 Late Chunking (2024 technique)

Traditional flow: text → chunks → embed each chunk. Problem: chunk ka embedding neighbors ka context miss karta hai.

Late chunking: Pehle *full document* ko long-context embedding model me daalo, token-level embeddings nikalo, *uske baad* pool karo chunk boundaries pe. Har chunk ka embedding global

context capture karta hai.

```
# Pseudo-code
token_embeds = long_context_encoder(full_doc) # contextualized
for chunk_span in chunk_boundaries:
    chunk_embed = mean_pool(token_embeds[chunk_span])
    store(chunk_text, chunk_embed)
```

- Models supporting this: jina-embeddings-v3, BGE-M3 with token-level access.

■ TIP

Late chunking se context dependency wali queries pe big wins milte hain — e.g., 'iska pricing kya hai?' jaha 'iska' previous paragraph se aata hai.

2.5 Propositional Chunking

Each chunk = ek atomic factual proposition. LLM se text ko discrete claims me todwao. Highly precise retrieval, but storage explodes (10x+ chunks per doc).

3. Query Transformations (Pre-Retrieval Magic)

User ki query directly embed karna often suboptimal hai. Pre-process karke better retrieval extract karo.

3.1 Query Rewriting

LLM use karke query ko rewrite karo — clearer, more specific, or in 'document language'.

```
PROMPT = '''Rewrite the following query to be more specific
and use formal language likely to appear in documentation.

Query: {query}
Rewritten:'''

# 'how do I cancel?' → 'What is the cancellation process for orders?'
```

3.2 Multi-Query Retrieval

Ek query se LLM se **multiple paraphrased versions** generate karo. Each pe retrieval karo. Results merge (deduplicate).

```
from langchain.retrievers.multi_query import MultiQueryRetriever

retriever = MultiQueryRetriever.from_llm(
    retriever=base_retriever,
    llm=ChatOpenAI(temperature=0)
)
# Behind the scenes:
# Query: 'GPU memory hierarchy?'
# Generated:
# 1. 'How is GPU memory organized?'
# 2. 'What are the levels of GPU memory?'
# 3. 'GPU memory architecture explained'
# Retrieves for each, combines unique results
```

- Embedding model ki blind spots cover karta hai
- Latency 3-5x, cost 3-5x

3.3 HyDE (Hypothetical Document Embeddings)

Brilliant idea: instead of embedding the *query*, embed a *hypothetical answer*.

```
Step 1: LLM se kaho — 'is query ka answer kya hoga? Ek paragraph likho.'
Step 2: Us hypothetical answer ko embed karo
Step 3: Real corpus me similar documents dhoondo

# Real answer documents milte hain → kyunki hypothetical answer
# real answer ke saath semantically similar hota hai (same 'space')
```

Kyun kaam karta hai: queries aur documents ka surface form different hota hai. 'GPU memory?' (short query) vs 'GPUs have a memory hierarchy consisting of...' (doc). Hypothetical answer document-style me hota hai → embedding alignment behtar.

```
HYDE_PROMPT = '''Write a concise paragraph that would answer:
{query}

Paragraph:'''

hyde_doc = llm.invoke(HYDE_PROMPT.format(query=q))
results = vectordb.similarity_search_by_vector(
    embedder.embed_query(hyde_doc.content), k=5
)
```

■■ WARNING

HyDE risky hai jab LLM out-of-domain hallucinate karta hai. Specialized domains (medical, legal) me carefully test karo.

3.4 Step-Back Prompting

Specific query se ek general/abstract version banao, dono pe retrieve karo. Useful jab specific facts surrounding general concepts par depend karte hain.

```
Original: 'What was Einstein's IQ?'
Step-back: 'What is known about Einstein's intelligence and biography?'

→ Both retrievals combined → richer context
```

3.5 Query Decomposition (Sub-query generation)

Complex multi-part queries ko sub-queries me todo. Each pe independent retrieval, results combine.

```
Query: 'Compare TPU v5 and H100 for transformer training'

Decomposed:
1. 'TPU v5 specs and performance'
2. 'H100 specs and performance'
3. 'Hardware considerations for transformer training'

→ 3 retrieval calls → merged context → comprehensive answer
```

3.6 Query Routing

Multiple data sources hain? LLM decide kare ki query kaha bhejni hai. Detailed coverage Part 3 (Agentic RAG) me — yaha brief intro.

- Vector store A (HR docs) vs Vector store B (Engineering docs) vs SQL database
- LLM classifier router decide karta hai destination
- Embedding-based router: query embedding vs each source's description embedding

4. Advanced Indexing Strategies

4.1 Parent-Child (Small-to-Big) Retrieval

Tension: chhote chunks → precise matching. Bade chunks → rich context. Solution: **match small, retrieve big**.

```
Indexing:
  Doc → Parent chunks (large, ~2000 tok)
      ■■→ Child chunks (small, ~200 tok)
  Embed children, store parents separately

Query time:
  Search against child embeddings
  Return corresponding PARENT chunk(s)
  LLM gets rich context, retrieval was precise
```

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore

parent_splitter = RecursiveCharacterTextSplitter(chunk_size=2000)
child_splitter = RecursiveCharacterTextSplitter(chunk_size=400)

retriever = ParentDocumentRetriever(
    vectorstore=vectordb,          # stores child vectors
    docstore=InMemoryStore(),      # stores parent docs
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)
retriever.add_documents(docs)
```

■ TIP

Parent-child production me dhamaal machata hai. Almost free quality boost over vanilla RAG. **Default banao apne pipeline me.**

4.2 Sentence Window Retrieval

Each sentence ek embedding banata hai (precise matching). Retrieval pe us sentence ke aas-paas ka window (say, ± 3 sentences) return karo.

```
# LlamaIndex example
from llama_index.core.node_parser import SentenceWindowNodeParser

parser = SentenceWindowNodeParser.from_defaults(
    window_size=3,
    window_metadata_key='window',
    original_text_metadata_key='original_text'
)
```

Similar idea to parent-child, but sliding window instead of hierarchy.

4.3 Auto-Merging Retrieval

Hierarchical chunks index karo. Query time: top-k child chunks le aao. Agar same parent ke multiple children retrieved hue (say, $\geq 50\%$), toh poora parent return karo.

- Lower-level chunks (leaves) zyada se zyada specific
- Multiple hits at same level → 'merge upward' to coarser chunk
- Best of granular matching + holistic context

4.4 Multi-Vector Retrievers

Ek document ke **multiple embeddings** store karo:

- **Embedding of full chunk** (vanilla)
- **Embedding of summary** (LLM-generated)
- **Embeddings of hypothetical questions** (LLM generates 'what questions does this answer?')
- **Embeddings of keywords/title**

Query pe sab embeddings se search; all paths same chunk pe converge. Recall dramatically improve.

```
# Generating hypothetical questions for each chunk
QGEN_PROMPT = '''Generate 3 questions this passage could answer:
{chunk}

Questions:'''

for chunk in chunks:
    questions = llm.invoke(QGEN_PROMPT.format(chunk=chunk.text))
    for q in parse(questions):
        vectordb.add(embed(q), payload={'chunk_id': chunk.id})
    # Original chunk stored separately in docstore
```

4.5 Summary Indexing

Long documents ke liye, full doc + summary dono store karo. Query first summaries se match karti hai → relevant docs identify → full content return.

Useful jab queries 'is doc me kya hai' type ki hain vs 'specific fact'.

5. Hybrid Search (Dense + Sparse)

Pure dense (embedding) retrieval ki blind spots hain — exact terms, IDs, codes, acronyms, rare words. Sparse retrieval (BM25) yaha shine karta hai. **Hybrid = both combined.**

5.1 BM25 Quick Refresher

BM25 = improved TF-IDF. Term frequency × inverse document frequency, with length normalization. 1990s ka algorithm, lekin abhi bhi production-grade IR me dominant.

```
score(q, d) = Σ IDF(q■) · [f(q■, d) · (k■ + 1)] /
               [f(q■, d) + k■ · (1 - b + b · |d|/avgdl)]

k■ ≈ 1.2-2.0    (term frequency saturation)
b ≈ 0.75        (length normalization)
```

- Exact term match → high score
- Rare terms (IDF high) → contribute more
- Long docs penalized → fairer scoring

5.2 Why Combine?

Query type	Dense wins	Sparse wins
'Tell me about transformers'	✓ (concept match)	
'GPT-4 vs Claude 3.5 Sonnet'	Partial	✓ (model names)
'Error code E_INVALID_TXN'		✓ (exact code)
'how to cancel my subscription'	✓ (paraphrase)	
'CVE-2024-1234'		✓ (exact ID)

5.3 Reciprocal Rank Fusion (RRF)

Sabse robust fusion technique. Multiple ranked lists ke ranks combine karta hai — scores ka raw value matter nahi karta (which is great because dense + BM25 scores incomparable hain).

```
RRF_score(d) = Σ 1 / (k + rank_i(d))
               i

k ≈ 60    (smoothing constant, standard)
rank_i(d) = position of d in i-th ranked list (1-indexed)

def reciprocal_rank_fusion(results_lists, k=60):
    scores = {}
    for results in results_lists:
        for rank, doc in enumerate(results, start=1):
            scores[doc.id] = scores.get(doc.id, 0) + 1/(k+rank)
    return sorted(scores.items(),
                  key=lambda x: x[1], reverse=True)
```

5.4 LangChain Ensemble Retriever

```
from langchain.retrievers import EnsembleRetriever, BM25Retriever

bm25 = BM25Retriever.from_documents(docs)
bm25.k = 5

dense = vectordb.as_retriever(search_kwargs={'k': 5})

ensemble = EnsembleRetriever(
    retrievers=[bm25, dense],
    weights=[0.4, 0.6]      # tune on eval set
)
results = ensemble.invoke('GPU SM warp scheduling')
```

5.5 SPLADE: Learned Sparse Embeddings

SPLADE = best of both worlds. Sparse vectors (interpretable, exact match) *but learned* from a transformer. Each query/doc → high-dim sparse vector jisme important terms ki weights hoti hain — including **expanded** terms.

- Documents auto-expand: 'dog' wala doc bhi 'puppy', 'canine' me weights paaye
- Trained via masked language modeling + sparsity regularization
- Better than BM25 + competitive with dense, with sparse retrieval's efficiency
- Tools: pyserini, naver/splade GitHub, BGE-M3 includes SPLADE-style mode

5.6 ColBERT (Late Interaction)

Multi-vector approach. Har token ka separate embedding store hota hai. Query time: **MaxSim** over tokens. Very accurate but storage-heavy.

```
Standard dense: cos(embed(q_full), embed(d_full))
ColBERT:       $\sum_{q \blacksquare d} \max \cos(\text{embed}(q \blacksquare), \text{embed}(d \blacksquare))$ 
```

■ NOTE

ColBERT v2 (with compression) ne production viability dramatically improve ki. ColPali = ColBERT for visual docs (PDFs as images). Worth checking out.

6. Re-ranking: Post-Retrieval Boost

Initial retrieval top-50 chunks le aata hai. Inme se top-5 LLM ko bhejne hain. **Re-ranker** = ek more accurate but slower model jo in 50 ko re-score karta hai. Final precision dramatically improve.

6.1 Bi-encoder vs Cross-encoder

```
Bi-encoder (your embedding model):
embed(query) → q_vec      ■■
embed(doc)   → d_vec      ■■ cosine(q_vec, d_vec)
↑ FAST: precompute all d_vecs

Cross-encoder (re-ranker):
[CLS] query [SEP] doc [SEP] → transformer → score
↑ ACCURATE but SLOW: must run per (q, d) pair
```

Tradeoff: cross-encoders 10-100x slower but significantly more accurate. Tum bi-encoder se top-50 fetch karte ho (fast), cross-encoder se top-5 select karte ho (accurate).

6.2 Popular Re-rankers

Re-ranker	Type	Notes
bge-reranker-large	Cross-encoder, open	Strong baseline, multilingual variants
bge-reranker-v2-m3	Cross-encoder, open	Multilingual, long context
Cohere Rerank 3	API	Strong, supports 100+ languages
Jina Reranker v2	API + open	Multilingual, code-aware
mxbai-rerank-large-v1	Open	Top of public benchmarks
RankGPT / RankLLM	LLM-as-reranker	Use GPT-4 to rerank — expensive but flexible
ColBERT	Late-interaction	Combined retrieval + reranking

6.3 Code Example: BGE Reranker

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder('BAAI/bge-reranker-large')

# Step 1: initial retrieval, fetch 30
candidates = vectordb.similarity_search(query, k=30)

# Step 2: rerank
pairs = [(query, doc.page_content) for doc in candidates]
scores = reranker.predict(pairs)

# Step 3: take top 5
ranked = sorted(zip(candidates, scores),
                 key=lambda x: x[1], reverse=True)
top5 = [doc for doc, _ in ranked[:5]]
```

6.4 LLM-as-Reranker

GPT-4/Claude ko directly score karne ke liye use karo. Slow + expensive, but very flexible — domain-specific criteria provide kar sakte ho prompt me.

```
RERANK_PROMPT = '''Rate the relevance of this passage to the query
on a scale of 0-10. Consider both topical match and information value.

Query: {query}
Passage: {passage}

Score (0-10):'''

# Batch inference for efficiency
```

■ KEY POINT

Re-ranking single biggest accuracy win in most RAG systems hai. Vanilla retrieve top-5 vs retrieve top-30 + rerank top-5 — second wins almost always. Latency cost ~50-200ms with hosted re-rankers. Add it.

7. Context Compression & Ordering

7.1 The Problem

Retrieved chunks me *relevant sentences* aur *filler* dono hote hain. Filler context window me space waste karta hai aur LLM ko distract karta hai. Compression = chunks se sirf relevant pieces extract karna.

7.2 LLMChainExtractor

LLM ko har chunk pe run karke sirf relevant sentences extract karwao.

```
from langchain.retrievers.document_compressors import LLMChainExtractor
from langchain.retrievers import ContextualCompressionRetriever

compressor = LLMChainExtractor.from_llm(ChatOpenAI())
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=base_retriever
)
```

- Drastically reduces context size
- Extra LLM call per chunk → latency + cost

7.3 EmbeddingsFilter

Cheap alternative: each chunk's similarity to query check karo; threshold ke neeche drop karo.

```
from langchain.retrievers.document_compressors import EmbeddingsFilter

filter = EmbeddingsFilter(embeddings=embeddings,
                          similarity_threshold=0.76)
# Bas relevant chunks pass through
```

7.4 LongLLMLingua

Research-grade prompt compression. Small LM use karta hai to identify token-level importance, redundant tokens drop karta hai. Up to 20x compression, minimal quality loss claim karta hai.

7.5 Lost-in-the-Middle Reordering

Famous finding (Liu et al., 2023): LLMs context ke **start** aur **end** ki info best attend karte hain. Middle 'forgotten' ho jata hai. Even strong models like GPT-4.

```
Solution: top-ranked chunk → END pe rakho (LLM attends to it most)
          2nd-ranked      → START pe rakho
          weakest         → MIDDLE
```

```
Pattern: [#2, #4, #6, ..., #5, #3, #1]
```

```
from langchain_community.document_transformers import LongContextReorder
```

```
reorder = LongContextReorder()  
reordered_docs = reorder.transform_documents(ranked_docs)
```

■ **TIP**

Lost-in-the-middle reordering is essentially free — no API calls, no models. Add it in production. Especially impactful when retrieving 8+ chunks.

8. Self-RAG & Corrective RAG

8.1 Self-RAG (Asai et al., 2023)

Vanilla RAG always retrieves. But sometimes retrieval not needed (general knowledge), sometimes retrieved chunks bad hain. Self-RAG fine-tuned model ko teach karta hai **special reflection tokens** generate karne ke liye:

Token	Meaning
[Retrieve]	Should I retrieve? Yes / No / Continue
[ISREL]	Is this passage relevant?
[ISSUP]	Does the passage support my answer?
[ISUSE]	How useful is my final answer (1-5)?

User: What's the capital of France?
Model: [Retrieve: No] → 'Paris is the capital of France.'

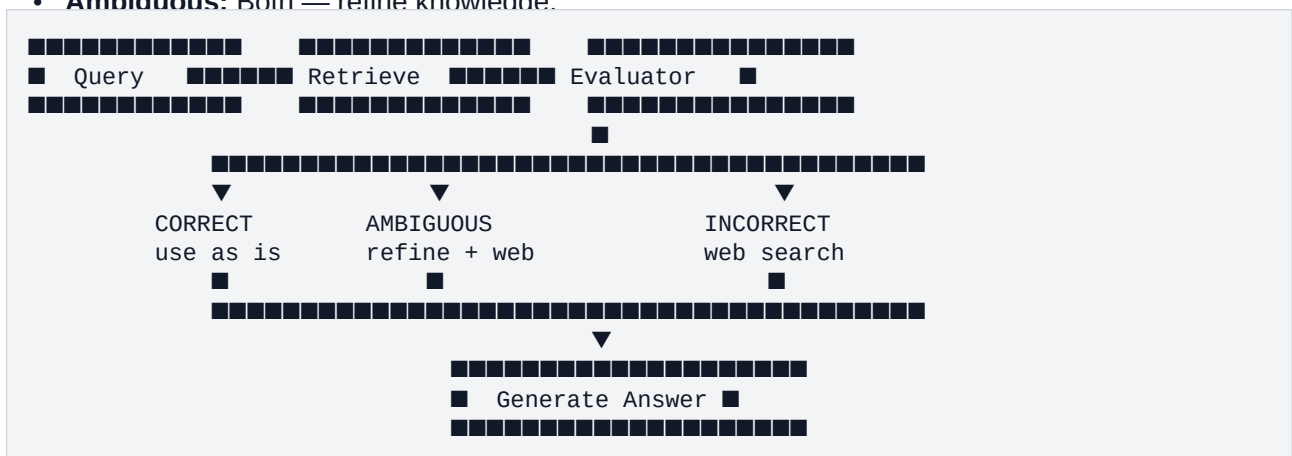
User: Latest revenue of Tata 1mg?
Model: [Retrieve: Yes] → search → [ISREL: Relevant]
→ generate → [ISSUP: Fully supported] [ISUSE: 5]

- Trained with synthetic data + critic LM annotations
- Open implementations available: selfrag/selfrag_llama2_7b

8.2 Corrective RAG (CRAG)

Lightweight idea, framework-friendly. After retrieval, ek **retrieval evaluator** classifies each retrieved chunk into:

- **Correct:** Use it as-is.
- **Incorrect:** Discard. Fall back to web search.
- **Ambiguous:** Both — refine knowledge.



CRAG ka biggest advantage: **fallback to web search** when corpus inadequate. Hallucination protection.

Evaluator can be a fine-tuned small model (T5-based) or LLM-as-judge.

9. RAG-Fusion, GraphRAG & Friends

9.1 RAG-Fusion

Multi-Query Retrieval + RRF combined. Generate multiple query variants, retrieve for each, fuse with RRF. Cleaner version of multi-query.

```
Step 1: LLM generates 4 query variants
Step 2: Each variant → retrieval (top-10)
Step 3: 4 ranked lists → RRF fusion → final top-k
Step 4: LLM generates answer using fused context
```

9.2 GraphRAG

Microsoft Research ne 2024 me paper drop kiya. Idea: corpus se **knowledge graph** extract karo (entities + relations), aur queries ko graph traversal + vector search dono se solve karo.

Pipeline

- **Entity extraction:** LLM se chunks me se entities (people, places, orgs, concepts) aur unke beech relations nikalwao.
- **Graph construction:** Nodes = entities, edges = relations. NetworkX, Neo4j, etc.
- **Community detection:** Leiden algorithm chalao to find topical clusters.
- **Community summaries:** Each cluster ka LLM-generated summary.
- **Query time:** Local search (nearby entities) ya global search (community summaries) depending on question scope.

```
# Microsoft's graphrag (Python package)
pip install graphrag
python -m graphrag.index --root ./project
python -m graphrag.query \
    --root ./project --method global \
    "What are the main themes?"
```

When GraphRAG wins: multi-hop reasoning, 'big picture' questions, connecting facts across documents.

When NOT: simple factoid retrieval, single-doc queries. Overhead high.

9.3 LightRAG (2024)

HKU's lighter alternative to GraphRAG. Dual-level retrieval (low-level: entities, high-level: themes). Less compute than GraphRAG, similar quality on many benchmarks.

9.4 Knowledge Graph + Vector Hybrid

Production approach: Neo4j (or similar) me structured KG, alongside vector store. Cypher queries for structured, vector for semantic, results combined.

```
# Neo4j + vector + LLM
MATCH (p:Product {id: 'X'})-[:HAS_FEATURE]->(f:Feature)
```



```
RETURN f.description
```

```
+ vector search for unstructured product reviews
```

10. Multimodal RAG

Real-world corpora me text alone nahi hota — images, tables, charts, audio, video. Multimodal RAG inko first-class citizens treat karta hai.

10.1 Approaches

Approach A: Caption everything → text-only RAG

Images/tables/charts ko LLM (GPT-4V, Claude vision) se describe karwao. Captions text corpus me ingest. Standard pipeline use karo.

- Simple, works with existing infra
- Information loss in captioning

Approach B: Unified multimodal embeddings

Models like CLIP, SigLIP, ImageBind images aur text ko ek shared embedding space me encode karte hain. Direct cross-modal retrieval possible.

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('clip-ViT-B-32')
img_embed = model.encode(Image.open('chart.png'))
txt_embed = model.encode('quarterly revenue chart')
# Both in same 512-d space
```

- True multimodal retrieval — text query → relevant image
- CLIP-family weaker than text-only on text retrieval

Approach C: ColPali — visual retrieval for PDFs

2024 release. PDF pages ko *directly* images ke roop me treat karta hai, ColBERT-style late interaction late karta hai. Tables, charts, layout — sab preserve. Game-changer for document QA.

```
# Pseudo-flow
for page in pdf:
    page_image = render(page)
    patches = vision_encoder(page_image) # multi-vec
    store(patches, metadata={'page': page_num})

# Query time: text query → token-level vecs
# Late interaction: max-sim across patches and tokens
```

10.2 Generation Side: Vision-Language Models

Multimodal context retrieve kiya — ab generation me bhi multimodal LLM chahiye. GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro, Qwen-VL, LLaVA — sab image+text input accept karte hain.

10.3 Tables in RAG

- **Convert to markdown/HTML** — text-only RAG handle kar leta hai for small tables.
- **Convert to natural language** — LLM se 'describe this table' karwao.

- **Structured retrieval** — large tables ko SQL DB me daalo, Text-to-SQL agent use karo (Part 3).
- **Specialized tools**: TabbyAPI, TableFormer, Unstructured.io table parsing.

11. Long-Context vs RAG: The Debate

Gemini 1.5 (2M tokens), Claude (200K), GPT-4.1 (1M) — context windows huge ho gaye hain. Kya RAG obsolete hai? Short answer: **no**, but landscape shift hua hai.

11.1 Long-Context Strengths

- Small-to-medium corpus completely fit ho jata hai → no retrieval needed
- Multi-document synthesis (no info missed)
- Cross-document reasoning seamless
- No retrieval engineering overhead

11.2 Long-Context Weaknesses

- **Cost:** Per-query token cost massive (e.g., 500K tokens × \$5/M = \$2.50/query)
- **Latency:** Time-to-first-token grows with input size
- **Recall vs reasoning:** Studies show long-context models perform well on retrieval-like tasks (needle-in-haystack) but degrade on multi-hop reasoning over long contexts
- **Doesn't scale:** 10M document corpus still won't fit anywhere
- **No updates:** Adding info means re-stuffing entire context

11.3 Modern Hybrid Pattern

■ KEY POINT

RAG retrieves a large, high-recall context (e.g., top-50 chunks, ~30K tokens) → **long-context LLM generates answer**. Best of both: retrieval scales, long-context handles synthesis without aggressive top-k filtering.

This 'RAG-into-long-context' is what most production systems are converging to in 2026.

12. RAGAS Deep Dive

RAGAS = standard library for RAG evaluation, uses LLMs-as-judges. Free, open-source, framework-agnostic.

12.1 Core Metrics (Detailed)

Faithfulness

Question: *Generated answer me jo claims hain, kya retrieved context inhe support karta hai?*

1. LLM ne answer ke claims extract kiye (statements list)
2. Each claim → verify against context (yes / no)
3. faithfulness = (#supported) / (#total)

1.0 = no hallucination, 0.0 = entire answer ungrounded

Answer Relevancy

Question: *Answer asked query ke direct point pe hai?*

1. LLM se 'reverse-engineer' karwao: from answer, generate possible questions
 2. Cosine sim(original_query, generated_questions) → average
- 0.0 = totally off-topic, 1.0 = perfectly on-point

Context Precision

Retrieved contexts me se kitne actually useful the (ranked relevance)? Higher = less noise.

Context Recall

Ground truth answer ke saare facts retrieved context me hain? Higher = retrieval miss kam.

Context Entity Recall

Specific entities (names, dates) retrieved context me hain? Useful for fact-heavy domains.

Answer Correctness

Combination of factual + semantic similarity vs ground truth answer.

12.2 Usage

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall,
    answer_correctness
)
from datasets import Dataset

data = {
    'question': [...],
    'answer': [...],          # your RAG output
    'contexts': [[...]],      # retrieved chunks per question
    'ground_truth': [...]    # gold answers
}
dataset = Dataset.from_dict(data)

results = evaluate(dataset, metrics=[
    faithfulness, answer_relevancy,
    context_precision, context_recall,
    answer_correctness
])
print(results)
# {'faithfulness': 0.87, 'answer_relevancy': 0.92, ...}
```

12.3 Synthetic Test Set Generation

RAGAS me built-in test set generator hai — apne corpus se Q-A-context triples bana deta hai.

```
from ragas.testset.generator import TestsetGenerator
from ragas.testset.evolutions import simple, reasoning, multi_context

generator = TestsetGenerator.from_langchain(
    generator_llm=ChatOpenAI(model='gpt-4o-mini'),
    critic_llm=ChatOpenAI(model='gpt-4o'),
    embeddings=OpenAIEmbeddings()
)

testset = generator.generate_with_langchain_docs(
    docs, test_size=50,
    distributions={simple: 0.5, reasoning: 0.25, multi_context: 0.25}
)
```

12.4 Other Eval Frameworks

- **TruLens**: Production monitoring + eval. Strong UI.
- **Phoenix (Arize)**: Tracing + eval, dataset management.
- **LangSmith**: LangChain ecosystem, dataset + traces + eval.
- **DeepEval**: Unit-test style RAG/LLM testing.
- **BEIR, MTEB**: Retrieval-only standard benchmarks.

13. Production Considerations

13.1 Caching

- **Embedding cache:** Same text → same vector. Cache hits → no API call. Critical at scale. Redis, in-memory LRU, or DB-backed.
- **Retrieval cache:** Same query → same results (TTL-based). Helps for popular queries (homepage chatbot, etc.).
- **LLM response cache:** Exact-match or semantic-match (GPTCache, Anthropic prompt caching). Repeats kafi savings.
- **Prompt caching:** Anthropic/OpenAI ka feature — long system prompts/context across requests reuse. Often 50-90% cost reduction.

13.2 Latency Optimization

- **Parallelize:** Multi-query retrieval, multi-source retrieval, re-ranking — async/await use karo.
- **Streaming:** User ko 'thinking' instead of dead silence. SSE / WebSocket.
- **Smaller embeddings:** MRL-enabled models me dim truncate karke 2-4x faster search.
- **Quantize vector index:** PQ/SQ — 10x faster search, minor accuracy hit.
- **Distillation:** Smaller LLMs (8B) fine-tuned on domain often outperform frontier models on narrow tasks, with 10-50x lower latency.

13.3 Cost Optimization

- **Two-stage models:** Cheap model (haiku/flash/3.5-mini) for routing, filtering. Expensive model only for final synthesis.
- **Batch embedding APIs:** 50% cheaper typically. Async OK for indexing.
- **Self-hosted embeddings:** At ~1M docs/month, self-hosted BGE pays off vs OpenAI.
- **Context budget:** top-k = 4 vs 10 — 2.5x cost difference. Eval the tradeoff.

13.4 Monitoring & Observability

- **Trace every request:** Query → retrievals → prompt → response. LangSmith / Phoenix / OpenTelemetry.
- **Quality signals:** Thumbs up/down, copy events, follow-up questions (often = bad answer).
- **Drift detection:** Query embedding distribution change → ingest new content.
- **P50, P95, P99 latency:** Track each stage.
- **Cost per query, per user:** Don't get surprised by month-end bill.

13.5 Security & Privacy

- **Prompt injection:** Retrieved docs could contain malicious instructions. Sanitize, use spotlighting/delimiters, refuse to follow instructions from context.

- **PII leakage:** Don't index secrets. Redaction pipelines.
- **Access control:** Metadata filtering by user permissions. Critical for enterprise.
- **Data residency:** Local-only embedding models for sensitive data (no API egress).

14. Production RAG Checklist

Ye list le ke har RAG project pe walk through karo:

- ■ **Eval set built** (50-200 examples min) with ground truth
- ■ **Chunking strategy** matched to content type (semantic / structure-aware)
- ■ **Embedding model** chosen via MTEB + domain testing
- ■ **Hybrid search** enabled (dense + BM25 + RRF)
- ■ **Parent-child / Multi-vec retrieval** for context-rich retrieval
- ■ **Re-ranker** in pipeline (BGE / Cohere)
- ■ **Lost-in-the-middle reordering** for top-k passages
- ■ **Metadata filtering** hooked up (user, time, source)
- ■ **Prompt template** with grounding + refusal + citation instructions
- ■ **Temperature 0**, max_tokens set
- ■ **Caching** (embedding + retrieval + prompt)
- ■ **RAGAS metrics** tracked in CI
- ■ **Tracing** (LangSmith / Phoenix) in production
- ■ **Prompt injection** defenses (delimiters, refusal training)
- ■ **Access control** via metadata filters
- ■ **User feedback loop** (■/■, follow-up signals)

■ KEY POINT

Tumne advanced RAG cover kar liya. **Part 3 (Agentic RAG)** me jaayenge — jaha RAG ek static pipeline nahi rahta, balki ek *agent* ban jata hai jo reason karta hai, multiple tools call karta hai, iteratively refine karta hai, aur dynamic decisions leta hai.